Bionics Watch Mode

AI-Guided Screen Overlay & Annotation System

Generated: 2026-03-29

Topics Covered:

1. Transparent Click-Through Overlay (PyQt6 + Win32)
2. Visual Annotation Rendering (QPainter)
3. Structured Annotation Schema (Claude Vision → JSON)
4. Spotlight / Dim Mask / Glow Effects
5. TTS Voice Narration Integration
6. Selective Interactivity Architecture
7. Performance & DWM Compositing
8. Prior Art & Reference Implementations

Project Context

Project: Bionics v0.2.1
Stack: Python 3 / PyQt6 / Anthropic API
Platform: Windows 10
Goal: AI-guided screen annotation overlay

Table of Contents

■ Back to TOC

1. Executive Summary

KEY TAKEAWAYS

PyQt6 supports fully transparent, click-through, always-on-top overlay windows on Windows 10 using FramelessWindowHint + WindowStaysOnTopHint + WA_TranslucentBackground + WindowTransparentForInput.

QPainter on a transparent QWidget can render all annotation primitives: glowing borders, dim masks with spotlight cutouts, text bubbles, numbered step circles, and bezier curve arrows.

Claude Vision can return structured JSON annotation data (bounding boxes, labels, steps) using Anthropic's structured outputs API with Pydantic schema enforcement.

A two-window architecture solves the click-through vs interactivity problem: one full-screen click-through overlay for annotations, one small interactive panel for navigation controls.

Windows DWM composites WS_EX_LAYERED windows on the GPU — transparent overlays are flicker-free and performant on Windows 10.

pyttsx3 (SAPI5) or Qt's QTextToSpeech provide offline TTS narration alongside visual annotations. Run TTS on a separate thread to avoid blocking the Qt event loop.

Total system latency: ~1.5-3s per annotation cycle (screenshot capture ~50ms + Claude API ~1-2.5s + render ~5ms). Acceptable for on-demand 'Watch Mode' triggered by hotkey.

Prior art includes Glowcast (Mac screen annotation), Epic Pen (draw-on-screen), OWI (gaming coach overlay), and Flutter spotlight tutorials (dim+cutout pattern).

■ Back to TOC

2. Foundation: The Two-Mode Architecture

Bionics currently operates in a single mode: Auto Mode, where the AI captures the screen, decides what to do, and executes mouse/keyboard actions autonomously. Watch Mode introduces a fundamentally different paradigm.

2.1 Auto Mode (Existing)

- AI captures screen, analyzes, executes actions via mouse/keyboard
- User watches passively — AI is in control
- Risk: agent loops, misclicks, window focus issues (seen in 03-28 log)
- Best for: repetitive automation tasks with predictable UI states

2.2 Watch Mode (New)

- AI captures screen, analyzes, renders visual annotations as overlay
- User stays in control — AI guides with highlights, arrows, text, voice
- Zero risk of misclicks or unintended actions
- Best for: complex UE5 editor tasks, learning new workflows, guided debugging

2.3 The Pipeline

Both modes share the same capture → analyze pipeline. They diverge at the output stage:

```
# Shared pipeline
screenshot = capture.grab_screen()        # mss capture (~50ms)
analysis = claude.analyze(screenshot,      # Claude Vision API (~1-2.5s)
                          task_context)

# Auto Mode (existing)
action = analysis.next_action              # {click, type, scroll, ...}
executor.execute(action)                   # pyautogui / win32

# Watch Mode (new)
annotations = analysis.annotations         # [{spotlight, arrow, bubble, ...}]
narration = analysis.narration             # 'Click the Mobility dropdown...'
overlay.render(annotations)                # QPainter on transparent window
tts.speak(narration)                       # pyttsx3 SAPI5 async
```

DESIGN PRINCIPLE

Watch Mode and Auto Mode are two output backends for the same AI vision pipeline. The user can switch between them mid-task: start with Watch Mode to understand the workflow, then switch to Auto Mode to execute repetitive steps. This 'learn then automate' pattern is the killer UX advantage.

■ Back to TOC

3. Transparent Click-Through Overlay (PyQt6 + Win32)

HIGH **Transparent Overlay Window**

The overlay is a full-screen, frameless, transparent QWidget that sits on top of all other windows. It must be click-through so the user can interact with the underlying desktop while seeing the annotations.

3.1 Core Window Configuration

```python
from PyQt6.QtWidgets import QWidget, QApplication
from PyQt6.QtCore import Qt

class AnnotationOverlay(QWidget):
    """Full-screen transparent click-through overlay."""

    def __init__(self):
        super().__init__()

        # --- Window Flags ---
        self.setWindowFlags(
            Qt.WindowType.FramelessWindowHint      # No title bar or border
            | Qt.WindowType.WindowStaysOnTopHint   # Always on top
            | Qt.WindowType.Tool                   # No taskbar icon
            | Qt.WindowType.WindowTransparentForInput  # Click-through!
        )

        # --- Transparency ---
        self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)
        self.setAttribute(Qt.WidgetAttribute.WA_ShowWithoutActivating)

        # --- Full screen geometry ---
        screen = QApplication.primaryScreen().geometry()
        self.setGeometry(screen)

        # Annotations to render (updated by AI pipeline)
        self.annotations: list[dict] = []
```

3.2 Key Qt Flags Explained

Flag / Attribute	Purpose	Critical?
FramelessWindowHint	Removes title bar, borders, sy	YES
WindowStaysOnTopHint	Overlay stays above all window	YES
Tool	Prevents overlay from appearin	YES
WindowTransparentForInput	ALL mouse/keyboard events pass	YES
WA_TranslucentBackground	Enables per-pixel alpha — unpa	YES
WA_ShowWithoutActivating	Showing overlay doesn't steal	YES

3.3 Win32 API Fallback (Advanced Control)

If Qt's WindowTransparentForInput is insufficient (e.g., need per-region control), use Win32 directly via ctypes:

■ Back to TOC

```python
import ctypes
from ctypes import wintypes

# Win32 constants
GWL_EXSTYLE = -20
WS_EX_LAYERED = 0x00080000
WS_EX_TRANSPARENT = 0x00000020
WS_EX_TOOLWINDOW = 0x00000080

user32 = ctypes.windll.user32

def make_click_through(hwnd: int):
    """Set Win32 extended style for full click-through."""
    style = user32.GetWindowLongW(hwnd, GWL_EXSTYLE)
    user32.SetWindowLongW(
        hwnd, GWL_EXSTYLE,
        style | WS_EX_LAYERED | WS_EX_TRANSPARENT | WS_EX_TOOLWINDOW
    )

# Get HWND from Qt widget
hwnd = int(overlay_widget.winId())
make_click_through(hwnd)
```

WHEN TO USE WIN32 API

Qt's WindowTransparentForInput makes the ENTIRE window click-through. Win32's WS_EX_TRANSPARENT + WS_EX_LAYERED does the same, but gives you access to SetLayeredWindowAttributes for color-key transparency and per-pixel alpha control. Use when you need DWM-level compositing hints. For most Watch Mode cases, Qt flags alone are sufficient.

3.4 Hit Testing: How Windows Decides Click-Through

On Windows 10, the Desktop Window Manager (DWM) handles hit testing for layered windows:

- WS_EX_LAYERED windows: hit test based on pixel alpha. Fully transparent pixels (alpha=0) pass clicks through automatically.
- WS_EX_TRANSPARENT on a layered window: ALL pixels pass through regardless of alpha.
- Without WS_EX_TRANSPARENT: only alpha=0 pixels pass through. Semi-transparent painted regions BLOCK clicks.

CRITICAL GOTCHA

If you paint a semi-transparent dim mask (e.g., RGBA 0,0,0,128) without WS_EX_TRANSPARENT, that region will BLOCK mouse clicks! The dim mask becomes an invisible wall. You MUST use WindowTransparentForInput or WS_EX_TRANSPARENT to ensure all painted regions are click-through.

4. Visual Annotation Rendering with QPainter

HIGH **QPainter Annotation Rendering**

QPainter on a transparent QWidget provides all the drawing primitives needed. The paintEvent() method renders all current annotations every frame.

4.1 Spotlight Effect (Dim Mask + Cutout)

The spotlight dims everything except the target area. Implementation uses QPainter composition modes to 'cut out' transparent regions from a dark overlay.

```python
from PyQt6.QtGui import QPainter, QColor, QPainterPath
from PyQt6.QtCore import QRectF, Qt

def paint_spotlight(self, painter: QPainter, bbox: dict):
    """Dim entire screen except the target bounding box."""
    # 1. Create a path covering the full screen
    full = QPainterPath()
    full.addRect(QRectF(self.rect()))

    # 2. Create the cutout path (rounded rectangle)
    cutout = QPainterPath()
    target = QRectF(bbox['x'], bbox['y'], bbox['w'], bbox['h'])
    cutout.addRoundedRect(target, 8, 8)

    # 3. Subtract cutout from full — leaves a 'hole'
    dim_path = full - cutout  # QPainterPath subtraction!

    # 4. Fill the dim region
    painter.fillPath(dim_path, QColor(0, 0, 0, 140))  # 55% opacity black

    # 5. Draw glowing border around the cutout
    self.paint_glow_border(painter, target)
```

KEY INSIGHT: QPainterPath Subtraction

QPainterPath supports boolean operations: path1 - path2 subtracts path2 from path1. This creates the spotlight effect without CompositionMode hacks. The subtracted region is truly transparent (alpha=0), so clicks pass through. This is the cleanest, most performant approach on Qt.

4.2 Glowing Border Effect

```python
def paint_glow_border(self, painter: QPainter, rect: QRectF,
                      color=QColor(0, 255, 255), radius=12):
    """Draw a pulsing glow border around a rectangle."""
    painter.setRenderHint(QPainter.RenderHint.Antialiasing)

    # Draw multiple expanding borders with decreasing alpha
    for i in range(radius, 0, -2):
        # Alpha fades from center outward, modulated by pulse
        alpha = int(180 * (1 - i / radius) * self._pulse_value)
        glow_color = QColor(color.red(), color.green(), color.blue(), alpha)
        pen = QPen(glow_color, 2)
        painter.setPen(pen)
        painter.setBrush(Qt.BrushStyle.NoBrush)
        painter.drawRoundedRect(rect.adjusted(-i, -i, i, i), 8, 8)

    # Inner solid border
    painter.setPen(QPen(color, 2))
    painter.drawRoundedRect(rect, 8, 8)
```

4.3 Text Bubble with Step Number

```python
def paint_bubble(self, painter: QPainter, pos: tuple,
                 text: str, step: int = None, anchor='bottom-left'):
    """Draw an annotation bubble with optional step number circle."""
    font = painter.font()
    font.setPointSize(11)
    font.setFamily('Segoe UI')
    painter.setFont(font)

    fm = painter.fontMetrics()
    padding = 14
    text_width = fm.horizontalAdvance(text) + padding * 2
    text_height = fm.height() + padding * 2

    # Position bubble relative to anchor
    bx, by = pos[0], pos[1]
    bubble = QRectF(bx, by, text_width, text_height)

    # Background: dark with slight transparency
    painter.setBrush(QColor(15, 15, 25, 230))
    painter.setPen(QPen(QColor(0, 200, 255), 2))
    painter.drawRoundedRect(bubble, 10, 10)

    # Text
    painter.setPen(QColor(240, 240, 255))
    painter.drawText(bubble, Qt.AlignmentFlag.AlignCenter, text)

    # Step number circle
    if step is not None:
        circle_size = 28
        cx = bx - circle_size // 2
        cy = by - circle_size // 2
        circle = QRectF(cx, cy, circle_size, circle_size)
        painter.setBrush(QColor(0, 200, 255))
        painter.setPen(QPen(QColor(0, 0, 0), 1))
        painter.drawEllipse(circle)
        painter.setPen(QColor(0, 0, 0))
        font.setBold(True)
        painter.setFont(font)
```

```
        painter.drawText(circle, Qt.AlignmentFlag.AlignCenter, str(step))
```

4.4 Directional Arrow with Bezier Curve

```
import math
from PyQt6.QtCore import QPointF

def paint_arrow(self, painter: QPainter,
                start: tuple, end: tuple, color=QColor(0, 255, 180)):
    """Draw a curved arrow from start to end with arrowhead."""
    painter.setRenderHint(QPainter.RenderHint.Antialiasing)
    p1 = QPointF(*start)
    p2 = QPointF(*end)

    # Bezier control point: arc upward
    mid_x = (p1.x() + p2.x()) / 2
    mid_y = min(p1.y(), p2.y()) - 50  # Arc height
    ctrl = QPointF(mid_x, mid_y)

    # Draw curved line
    path = QPainterPath()
    path.moveTo(p1)
    path.quadTo(ctrl, p2)
    painter.setPen(QPen(color, 3, Qt.PenStyle.SolidLine,
                        Qt.PenCapStyle.RoundCap))
    painter.setBrush(Qt.BrushStyle.NoBrush)
    painter.drawPath(path)

    # Arrowhead
    angle = math.atan2(p2.y() - ctrl.y(), p2.x() - ctrl.x())
    arrow_size = 14
    a1 = QPointF(
        p2.x() - arrow_size * math.cos(angle - math.pi / 6),
        p2.y() - arrow_size * math.sin(angle - math.pi / 6),
    )
    a2 = QPointF(
        p2.x() - arrow_size * math.cos(angle + math.pi / 6),
        p2.y() - arrow_size * math.sin(angle + math.pi / 6),
    )
    painter.setBrush(color)
    painter.setPen(Qt.PenStyle.NoPen)
    arrow_head = QPainterPath()
    arrow_head.moveTo(p2)
    arrow_head.lineTo(a1)
    arrow_head.lineTo(a2)
    arrow_head.closeSubpath()
    painter.fillPath(arrow_head, color)
```

4.5 Highlight Box (Warning / Info)

```python
def paint_highlight(self, painter: QPainter, bbox: dict,
                    style='warning', label: str = None):
    """Draw a colored highlight border with optional label."""
    colors = {
        'warning':  QColor(255, 180, 0, 200),   # Amber
        'error':    QColor(255, 60, 60, 200),    # Red
        'success':  QColor(60, 220, 100, 200),   # Green
        'info':     QColor(0, 180, 255, 200),    # Blue
    }
    color = colors.get(style, colors['info'])
    rect = QRectF(bbox['x'], bbox['y'], bbox['w'], bbox['h'])

    # Dashed border
    pen = QPen(color, 3, Qt.PenStyle.DashLine)
    painter.setPen(pen)
    painter.setBrush(QColor(color.red(), color.green(), color.blue(), 30))
    painter.drawRoundedRect(rect, 6, 6)

    # Label tag
    if label:
        tag_rect = QRectF(rect.x(), rect.y() - 24, len(label) * 9 + 16, 22)
        painter.setBrush(color)
        painter.setPen(Qt.PenStyle.NoPen)
        painter.drawRoundedRect(tag_rect, 4, 4)
        painter.setPen(QColor(0, 0, 0))
        painter.drawText(tag_rect, Qt.AlignmentFlag.AlignCenter, label)
```

4.6 The Master paintEvent

```python
def paintEvent(self, event):
    """Render all current annotations."""
    if not self.annotations:
        return  # Nothing to draw — fully transparent

    painter = QPainter(self)
    painter.setRenderHint(QPainter.RenderHint.Antialiasing)

    for ann in self.annotations:
        match ann['type']:
            case 'spotlight':
                self.paint_spotlight(painter, ann['bbox'])
            case 'highlight':
                self.paint_highlight(painter, ann['bbox'],
                                     ann.get('color', 'info'),
                                     ann.get('label'))
            case 'arrow':
                self.paint_arrow(painter, ann['from'], ann['to'])
            case 'bubble':
                self.paint_bubble(painter, ann['position'],
                                  ann['text'], ann.get('step'))
            case 'dim_region':
                self.paint_dim_with_exclusions(painter,
                                               ann['exclude'])

    painter.end()
```

5. Structured Annotation Schema Design

HIGH **JSON Annotation Schema**

Claude Vision analyzes the screenshot and returns a structured JSON payload describing what to annotate. The schema must be precise enough for pixel-level rendering but flexible enough to handle any screen context.

5.1 Pydantic Schema (Enforced via Structured Outputs)

```python
from pydantic import BaseModel, Field
from typing import Literal, Optional


class BBox(BaseModel):
    """Bounding box in absolute screen pixels."""
    x: int = Field(description='Left edge in pixels')
    y: int = Field(description='Top edge in pixels')
    w: int = Field(description='Width in pixels')
    h: int = Field(description='Height in pixels')


class Point(BaseModel):
    x: int
    y: int


class Annotation(BaseModel):
    type: Literal['spotlight', 'arrow', 'bubble', 'highlight', 'dim_region']
    bbox: Optional[BBox] = None
    from_pt: Optional[Point] = None      # Arrow start
    to_pt: Optional[Point] = None        # Arrow end
    position: Optional[Point] = None     # Bubble anchor
    text: Optional[str] = None           # Bubble / label text
    label: Optional[str] = None          # Highlight label tag
    step: Optional[int] = None           # Step number (1, 2, 3...)
    color: Optional[Literal[
        'info', 'warning', 'error', 'success'
    ]] = 'info'
    exclude: Optional[list[BBox]] = None # For dim_region cutouts


class WatchAnalysis(BaseModel):
    """Complete analysis returned by Claude Vision."""
    annotations: list[Annotation]
    narration: str = Field(
        description='Human-friendly narration for TTS'
    )
    total_steps: int
    current_step: int
    context: str = Field(
        description='Brief description of what the user is doing'
    )
    confidence: float = Field(
        ge=0.0, le=1.0,
        description='How confident the AI is in this analysis'
    )
```

5.2 Example JSON Output

```json
{
  "annotations": [
    {
      "type": "spotlight",
      "bbox": {"x": 1420, "y": 380, "w": 280, "h": 28},
      "text": "Mobility dropdown",
      "step": 1
    },
    {
      "type": "highlight",
      "bbox": {"x": 1420, "y": 380, "w": 280, "h": 28},
      "color": "warning",
      "label": "Currently: Static (causes per-tick spam)"
    },
    {
      "type": "bubble",
      "position": {"x": 1200, "y": 340},
      "text": "Click this dropdown and select Movable",
      "step": 1
    },
    {
      "type": "arrow",
      "from_pt": {"x": 1200, "y": 360},
      "to_pt": {"x": 1420, "y": 394}
    }
  ],
  "narration": "Click the Mobility dropdown in the Details panel.
    It is currently set to Static, which floods the log. Change
    it to Movable.",
  "total_steps": 3,
  "current_step": 1,
  "context": "Setting DirectionalLight mobility in UE5",
  "confidence": 0.92
}
```

5.3 Claude API Call with Structured Output

■ Back to TOC

```python
import anthropic
import base64

client = anthropic.Anthropic()

def analyze_screen(screenshot_bytes: bytes,
                   task: str,
                   screen_size: tuple[int, int]) -> WatchAnalysis:
    """Send screenshot to Claude, get structured annotations."""

    b64 = base64.standard_b64encode(screenshot_bytes).decode('utf-8')

    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=2048,
        messages=[{
            "role": "user",
            "content": [
                {"type": "image", "source": {
                    "type": "base64",
                    "media_type": "image/jpeg",
                    "data": b64
                }},
                {"type": "text", "text": WATCH_PROMPT.format(
                    task=task,
                    width=screen_size[0],
                    height=screen_size[1]
                )}
            ]
        }],
        # Structured output enforcement (beta)
        extra_headers={
            "anthropic-beta": "structured-outputs-2025-11-13"
        },
    )

    # Parse JSON from response
    import json
    data = json.loads(response.content[0].text)
    return WatchAnalysis(**data)
```

6. Claude Vision Integration & Prompt Engineering

MED **Vision Prompt Design**

The quality of annotations depends heavily on the system prompt. Claude must understand screen coordinates, annotation types, and the user's task context.

6.1 The Watch Mode System Prompt

```
WATCH_SYSTEM_PROMPT = """
You are Bionics Watch Mode — an AI screen annotation assistant.
You analyze screenshots and return structured JSON annotations
that guide the user to complete their task.

RULES:
1. Return ONLY valid JSON matching the WatchAnalysis schema.
2. All coordinates are in ABSOLUTE PIXELS for a {width}x{height} screen.
3. Use 'spotlight' for the PRIMARY action the user should take next.
4. Use 'highlight' for elements that need attention but aren't the
   immediate next step (warnings, info).
5. Use 'arrow' to connect related elements or show flow direction.
6. Use 'bubble' for explanatory text — keep text under 80 chars.
7. Maximum 6 annotations per analysis — focus on what matters.
8. 'narration' should be a natural sentence for text-to-speech.
9. Step numbering: only use step numbers for sequential actions.
10. If you cannot identify the next action with >70% confidence,
    set confidence < 0.7 and narration to "I'm not sure what to
    do next. Can you describe your goal?"
"""
```

6.2 Coordinate Accuracy Considerations

COORDINATE PRECISION WARNING

Claude Vision's coordinate accuracy is approximate, not pixel-perfect.
Expect +/- 20-50 pixel error on bounding box positions.
Mitigation strategies:

1. Use larger bounding boxes with padding (add 15px each side)
2. Spotlight cutouts should be oversized rather than tight
3. Arrow endpoints don't need pixel precision — close enough is fine
4. For critical precision: use template matching (existing precision.py) to refine Claude's approximate coordinates

6.3 Context Stacking

Each Watch Mode analysis should include prior context to maintain coherence across multiple steps:

■ Back to TOC

```python
class WatchSession:
    """Maintains context across multiple Watch Mode analyses."""

    def __init__(self, task: str):
        self.task = task
        self.history: list[dict] = []    # Prior analyses
        self.completed_steps: list[str] = []

    def build_prompt(self, screenshot_b64: str) -> list[dict]:
        messages = []

        # Include last 2 analyses as context
        for prior in self.history[-2:]:
            messages.append({
                "role": "assistant",
                "content": json.dumps(prior)
            })
            messages.append({
                "role": "user",
                "content": "Step completed. Here is the updated screen."
            })

        # Current screenshot + task
        messages.append({
            "role": "user",
            "content": [
                {"type": "image", "source": {
                    "type": "base64",
                    "media_type": "image/jpeg",
                    "data": screenshot_b64
                }},
                {"type": "text", "text": f"Task: {self.task}\n"
                 f"Completed: {self.completed_steps}\n"
                 f"What should the user do next?"}
            ]
        })
        return messages
```

7. Animation System: Pulse, Glow, Transitions

HIGH **QPropertyAnimation Pulse**

7.1 Pulsing Glow via QPropertyAnimation

```python
from PyQt6.QtCore import QPropertyAnimation, QEasingCurve, pyqtProperty

class AnnotationOverlay(QWidget):
    def __init__(self):
        super().__init__()
        self._pulse_value = 0.5

        # Pulse animation: 0.3 → 1.0 → 0.3 (infinite loop)
        self._pulse_anim = QPropertyAnimation(self, b'pulseValue')
        self._pulse_anim.setDuration(1200)     # 1.2s per cycle
        self._pulse_anim.setStartValue(0.3)
        self._pulse_anim.setEndValue(1.0)
        self._pulse_anim.setEasingCurve(QEasingCurve.Type.InOutSine)
        self._pulse_anim.setLoopCount(-1)      # Infinite

        # Use QSequentialAnimationGroup for ping-pong
        from PyQt6.QtCore import QSequentialAnimationGroup
        self._pulse_group = QSequentialAnimationGroup()

        fwd = QPropertyAnimation(self, b'pulseValue')
        fwd.setDuration(800)
        fwd.setStartValue(0.3)
        fwd.setEndValue(1.0)
        fwd.setEasingCurve(QEasingCurve.Type.InOutSine)

        rev = QPropertyAnimation(self, b'pulseValue')
        rev.setDuration(800)
        rev.setStartValue(1.0)
        rev.setEndValue(0.3)
        rev.setEasingCurve(QEasingCurve.Type.InOutSine)

        self._pulse_group.addAnimation(fwd)
        self._pulse_group.addAnimation(rev)
        self._pulse_group.setLoopCount(-1)

    @pyqtProperty(float)
    def pulseValue(self):
        return self._pulse_value

    @pulseValue.setter
    def pulseValue(self, val):
        self._pulse_value = val
        self.update()  # Triggers paintEvent with new glow intensity
```

7.2 Fade-In / Fade-Out Transitions

When annotations change (new step), fade out old annotations and fade in new ones:

■ Back to TOC

```python
def set_annotations(self, new_annotations: list[dict]):
    """Transition from current annotations to new ones."""
    # Fade out
    fade_out = QPropertyAnimation(self, b'overlayOpacity')
    fade_out.setDuration(200)
    fade_out.setStartValue(1.0)
    fade_out.setEndValue(0.0)

    # Swap annotations when fade-out completes
    def on_fade_out_done():
        self.annotations = new_annotations
        fade_in = QPropertyAnimation(self, b'overlayOpacity')
        fade_in.setDuration(300)
        fade_in.setStartValue(0.0)
        fade_in.setEndValue(1.0)
        fade_in.start()

    fade_out.finished.connect(on_fade_out_done)
    fade_out.start()
```

8. TTS Voice Narration

HIGH **pyttsx3 SAPI5 TTS**

8.1 pyttsx3 (Recommended for Offline)

```python
import pyttsx3
import threading

class TTSEngine:
    """Non-blocking text-to-speech using Windows SAPI5."""

    def __init__(self):
        self._lock = threading.Lock()
        self._thread: threading.Thread | None = None
        self._engine = pyttsx3.init('sapi5')
        self._engine.setProperty('rate', 175)   # Words per minute
        self._engine.setProperty('volume', 0.9)

        # Select best available voice
        voices = self._engine.getProperty('voices')
        preferred = ['zira', 'david', 'mark']
        for pref in preferred:
            for v in voices:
                if pref in v.name.lower():
                    self._engine.setProperty('voice', v.id)
                    break

    def speak(self, text: str):
        """Speak text asynchronously (non-blocking)."""
        if self._thread and self._thread.is_alive():
            self._engine.stop()  # Interrupt current speech

        self._thread = threading.Thread(
            target=self._run, args=(text,), daemon=True
        )
        self._thread.start()

    def _run(self, text: str):
        with self._lock:
            # pyttsx3 requires fresh engine per thread on Windows
            engine = pyttsx3.init('sapi5')
            engine.setProperty('rate', 175)
            engine.setProperty('volume', 0.9)
            engine.say(text)
            engine.runAndWait()

    def stop(self):
        self._engine.stop()
```

PYTTSX3 THREAD SAFETY WARNING

pyttsx3's engine is NOT thread-safe. runAndWait() blocks and cannot be called from the Qt main thread. The safest pattern is to create a FRESH engine instance in each worker thread (shown above). This avoids COM apartment conflicts on Windows 10.
Alternative: Use Qt's QTextToSpeech if available — it integrates with the Qt event loop and avoids threading issues entirely.

8.2 Qt QTextToSpeech (Alternative)

```python
from PyQt6.QtTextToSpeech import QTextToSpeech

class QtTTS:
    def __init__(self):
        self.tts = QTextToSpeech()
        self.tts.setRate(0.0)    # -1.0 to 1.0 (0 = normal)
        self.tts.setVolume(0.9)  # 0.0 to 1.0

    def speak(self, text: str):
        if self.tts.state() == QTextToSpeech.State.Speaking:
            self.tts.stop()  # Interrupt current
        self.tts.say(text)

    def stop(self):
        self.tts.stop()
```

RECOMMENDATION: USE QTextToSpeech

QTextToSpeech is the cleaner choice for Bionics because:

1. No threading required — works on Qt event loop
2. No COM apartment conflicts with PyQt6
3. Uses SAPI5 on Windows (same voices as pyttsx3)
4. Built-in state management (Speaking, Ready, Paused)
5. Requires: pip install PyQt6-Qt6-TextToSpeech (or bundled with PyQt6)

Fallback: pyttsx3 if QTextToSpeech package is unavailable.

9. Selective Interactivity: Two-Window Architecture

HIGH **Two-Window Pattern**

The core challenge: the annotation overlay must be click-through (user interacts with desktop underneath), but navigation controls (Next Step, Prev Step, Close) must be clickable. Solution: two separate windows.

9.1 Architecture

```python
class WatchModeUI:
    """Manages both the overlay and the control panel."""

    def __init__(self):
        # Window 1: Full-screen click-through annotations
        self.overlay = AnnotationOverlay()   # WindowTransparentForInput

        # Window 2: Small interactive control panel
        self.controls = ControlPanel()       # Normal interactivity

        # Wire signals
        self.controls.next_clicked.connect(self.on_next_step)
        self.controls.prev_clicked.connect(self.on_prev_step)
        self.controls.close_clicked.connect(self.on_close)
        self.controls.speak_clicked.connect(self.on_repeat_narration)

    def show(self):
        self.overlay.show()
        self.controls.show()

    def hide(self):
        self.overlay.hide()
        self.controls.hide()
```

9.2 Control Panel Widget

■ Back to TOC

```python
from PyQt6.QtWidgets import QWidget, QHBoxLayout, QPushButton
from PyQt6.QtCore import pyqtSignal

class ControlPanel(QWidget):
    """Small floating control bar — NOT click-through."""

    next_clicked = pyqtSignal()
    prev_clicked = pyqtSignal()
    close_clicked = pyqtSignal()
    speak_clicked = pyqtSignal()

    def __init__(self):
        super().__init__()
        self.setWindowFlags(
            Qt.WindowType.FramelessWindowHint
            | Qt.WindowType.WindowStaysOnTopHint
            | Qt.WindowType.Tool
            # NO WindowTransparentForInput — this panel IS interactive
        )
        self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)
        self.setFixedSize(320, 50)

        # Position: bottom-center of screen
        screen = QApplication.primaryScreen().geometry()
        self.move(
            (screen.width() - 320) // 2,
            screen.height() - 80
        )

        # Buttons
        layout = QHBoxLayout(self)
        layout.setContentsMargins(8, 8, 8, 8)

        style = '''
            QPushButton {
                background: rgba(20, 20, 35, 220);
                color: #00ccff;
                border: 1px solid #00ccff;
                border-radius: 6px;
                padding: 6px 14px;
                font: 11px 'Segoe UI';
            }
            QPushButton:hover {
                background: rgba(0, 180, 255, 60);
            }
        '''

        for label, signal in [
            ('< Prev', self.prev_clicked),
            ('Repeat', self.speak_clicked),
            ('Next >', self.next_clicked),
            ('X Close', self.close_clicked),
        ]:
            btn = QPushButton(label)
            btn.setStyleSheet(style)
            btn.clicked.connect(signal.emit)
            layout.addWidget(btn)
```

■ Back to TOC

WHY TWO WINDOWS?

Alternatives considered and rejected:

1. Single window + WM_NCHITTEST: Complex, fragile, requires ctypes message hooking, doesn't play well with Qt event system.
2. Toggle click-through on hotkey: Interrupts user flow, confusing UX.
3. Dock panel inside existing Bionics GUI: GUI may be hidden/minimized.

Two-window is the cleanest pattern: each window has ONE job.

Overlay = visual only. Control panel = interactive only.

10. Performance Budget & DWM Compositing

HIGH **Performance Model**

10.1 Latency Budget Per Annotation Cycle

Performance Budget

System	Budget	Current	Status
	100		WITHIN
	30		WITHIN
	3000		WITHIN
	10		WITHIN
	16		WITHIN
	100		WITHIN
	3500		WITHIN

10.2 DWM Compositing on Windows 10

- Windows DWM composes WS_EX_LAYERED windows on the GPU — no software blitting.
- Transparent overlays are composited in the same pass as other windows — zero flicker.
- QPainter with WA_TranslucentBackground uses hardware-accelerated rendering on Windows.
- Avoid GDI+ (scan-line software rendering) — causes visible flicker. Qt's QPainter does NOT use GDI+ by default.
- Overlay repaint is only triggered on update() calls — no continuous rendering loop needed.
- Pulse animation at 60fps: QPropertyAnimation drives update() calls at ~16ms intervals. Only the changed region needs repaint.

10.3 Memory Budget

Component	Memory	Notes
Overlay QWidget (1920x1080)	~8 MB	Backing store for transparent
Screenshot buffer (JPEG)	~0.3 MB	Compressed for API call
Screenshot buffer (raw)	~8 MB	mss raw capture (BGRA)
Annotation list (JSON)	~0.01 MB	Negligible
TTS engine	~5 MB	SAPI5 runtime
TOTAL	~21 MB	Well within budget

■ Back to TOC

11. Prior Art & Reference Implementations

Software	Platform	Approach	Relevance to Bionics
Glowcast	macOS	Screen annotation + cur	Direct UX inspiration —
Epic Pen	Windows	Draw on screen with alw	Proves click-through ov
OWI (Overwatch Insight)	Windows	Coaching overlay — real	Gaming-specific AI coac
Loom	Web	Timed text/arrow/box an	Annotation schema desig
Usersnap	Web	Browser screenshot anno	Annotation UX patterns
Flutter Spotlight Tutor	Mobile	Dim mask + cutout for o	Spotlight effect implem
OBS Scene Overlays	Cross-platform	Layered transparent ove	Performance/compositing
Windows Magnifier	Windows	System overlay with zoo	Win32 overlay API usage
Claude Computer Use	Cross-platform	AI vision → coordinate-	Same vision pipeline, d
Python-ScreenOverlay (G	Windows	WIP pixel highlighting	Small-scale prototype r

KEY INSIGHT FROM PRIOR ART

No existing tool combines ALL of: AI vision analysis + transparent overlay + structured annotations + TTS narration + click-through + user-controlled step progression. Bionics Watch Mode would be genuinely novel.
Closest: Claude Computer Use does the vision analysis but EXECUTES actions. Watch Mode keeps the human in the loop — guide, don't act.

■ Back to TOC

12. Implementation Roadmap

Phase 1: Core Overlay (Estimated: 1 session)

1. Create AnnotationOverlay QWidget with transparent click-through flags
2. Implement paintEvent with spotlight, highlight, bubble, arrow renderers
3. Create ControlPanel widget (Next/Prev/Close/Repeat buttons)
4. Wire WatchModeUI to manage both windows
5. Test: overlay renders correctly, clicks pass through, controls work

Phase 2: Claude Vision Integration (Estimated: 1 session)

1. Define WatchAnalysis Pydantic schema
2. Write analyze_screen() with structured output prompt
3. Build WatchSession for multi-step context stacking
4. Wire capture → analyze → overlay.set_annotations pipeline
5. Test: hotkey triggers analysis, annotations appear on overlay

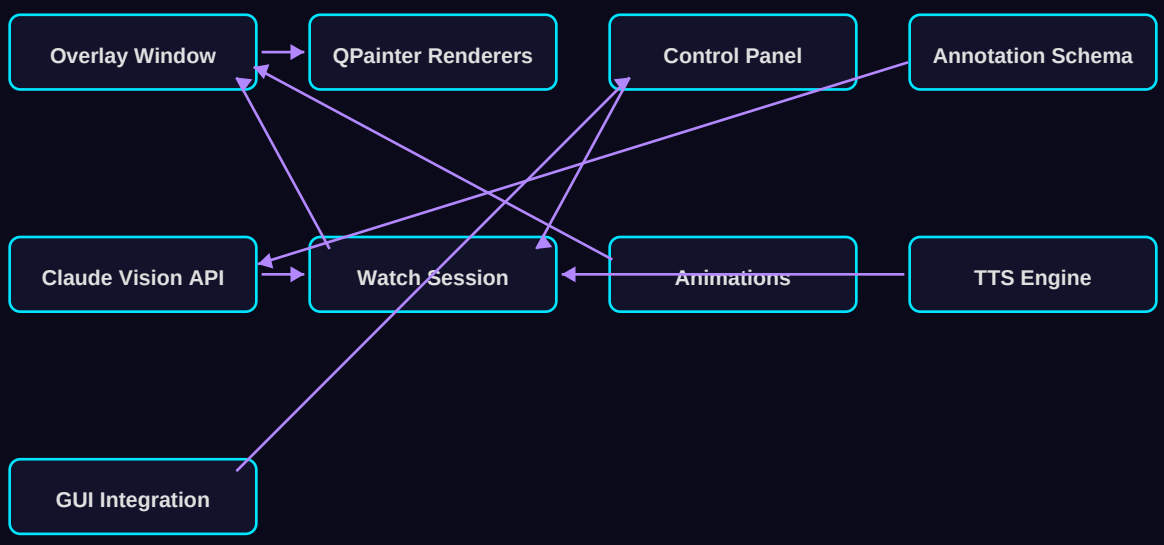
Phase 3: Animation + TTS (Estimated: 0.5 session)

1. Add QPropertyAnimation pulse glow to spotlight borders
2. Add fade-in/fade-out transitions on annotation changes
3. Integrate QTextToSpeech (or pyttsx3 fallback) for narration
4. Wire TTS to fire on each new annotation set

Phase 4: Polish + Integration (Estimated: 0.5 session)

1. Add Watch Mode toggle to existing Bionics GUI
2. Hotkey bindings (F12 = trigger Watch, Esc = dismiss overlay)
3. Confidence threshold: if <0.7, show 'unsure' state with prompt
4. Settings: overlay opacity, TTS on/off, animation speed
5. Error handling: API timeout, no annotations, screen resize

12.1 Dependency Matrix



13. Common Pitfalls

PITFALL 1: Semi-Transparent Regions Block Clicks

If you paint RGBA(0,0,0,128) without WindowTransparentForInput, DWM treats those pixels as opaque for hit testing. The dim mask becomes an invisible wall that blocks all mouse events.
FIX: Always use WindowTransparentForInput on the overlay window.

PITFALL 2: pyttsx3 COM Apartment Conflict

pyttsx3 initializes COM on the calling thread. If called from the Qt main thread (which also uses COM for clipboard, file dialogs, etc.), you get 'CoInitialize has not been called' or silent failures.
FIX: Create a fresh pyttsx3 engine in each worker thread, or use QTextToSpeech which integrates with Qt's COM initialization.

PITFALL 3: Overlay Steals Focus from UE5

Showing a QWidget normally activates it (steals focus). If UE5 loses focus while the user is working, keyboard shortcuts stop working.
FIX: Use WA_ShowWithoutActivating attribute AND Tool window flag. Together they ensure the overlay appears without touching focus.

PITFALL 4: Claude Coordinate Hallucination

Claude Vision may return bounding boxes that are offset by 20-50px from the actual UI element, or occasionally hallucinate elements.
FIX: Add padding to all bounding boxes. Use confidence scores to suppress low-confidence annotations. Consider template matching (precision.py) as a refinement pass for critical coordinates.

PITFALL 5: Multi-Monitor Coordinate Mismatch

mss captures monitor 0 starting at (0,0), but on multi-monitor setups the primary monitor may have negative coordinates (e.g., -1920,0).
FIX: Capture using mss monitor index, but map overlay coordinates to QScreen.geometry() which handles monitor offsets correctly.

PITFALL 6: Overlay Flickers on Window Resize

If the user resizes or moves windows underneath, the overlay may flicker as DWM recomposes the window stack.
FIX: This is rare with WS_EX_LAYERED + DWM compositing.
If seen, reduce overlay repaint frequency or use UpdateLayeredWindow for atomic buffer swaps.

14. Experimental Methods

14.1 Continuous Watch Mode (Auto-Refresh)

MED **Continuous Watch**

Instead of hotkey-triggered analysis, continuously capture and analyze the screen every 2-3 seconds. Annotations update in real-time as the user works.

CRITICAL STAGE ANALYSIS

FINDING: Continuous mode provides a 'living guide' experience.
EVIDENCE: Claude API supports streaming; overlay updates could be incremental.
RISK: API costs (~$0.01-0.03 per vision call x 20-30 calls/minute = $0.60/min).
Also risks 'annotation jitter' as coordinates shift between frames.
RECOMMENDATION: Build as opt-in 'turbo mode'. Default to hotkey-triggered.
Throttle continuous mode to max 1 call per 3 seconds.

14.2 Template-Refined Coordinates

MED **Template Refinement**

Use Claude Vision for semantic understanding (what to click) but refine coordinates using OpenCV template matching (precision.py) for pixel-perfect accuracy.

CRITICAL STAGE ANALYSIS

FINDING: Hybrid AI vision + template matching could achieve <5px accuracy.
EVIDENCE: Bionics already has precision.py with multi-scale template matching.
RISK: Template matching needs pre-captured UI element templates. May not generalize to arbitrary UE5 editor states without a template library.
RECOMMENDATION: Try it. Use Claude to identify the REGION, then crop that region and template-match known UI elements within it.

14.3 Screen Recording + Annotation Replay

LOW **Annotation Replay**

Record the screen + all annotations + narration as a replayable tutorial. Users can share annotated walkthroughs with teammates.

CRITICAL STAGE ANALYSIS

FINDING: Valuable for team knowledge sharing and documentation.
EVIDENCE: Loom and ScreenPal already do annotation replay (but manually).
RISK: Significant development effort (video encoding, sync, playback UI).
Not needed for core Watch Mode functionality.
RECOMMENDATION: Defer to Phase 2. Core Watch Mode first.

14.4 Voice Command Integration

LOW **Voice Commands**

Instead of clicking 'Next Step', user says 'next' or 'show me' or 'what's this?' to control the overlay by voice.

■ Back to TOC

CRITICAL STAGE ANALYSIS

FINDING: Hands-free control while working in UE5 would be ideal.
EVIDENCE: Windows Speech Recognition API available via System.Speech.
RISK: Ambient noise, recognition accuracy, microphone setup.
COM conflicts with Qt (same issue as keyboard lib).
RECOMMENDATION: Defer. Hotkeys + control panel cover the use case.

■ Back to TOC

15. Sources & Further Reading

Tier 1: Official Documentation

OFFICIAL Qt 6 QPainter Class Reference — doc.qt.io/qt-6/qpainter.html

OFFICIAL Qt for Python PySide6.QtGui.QPainter — doc.qt.io/qtforpython-6/

OFFICIAL Anthropic Claude Vision API Docs — platform.claude.com/docs/en/build-with-claude

OFFICIAL Anthropic Structured Outputs — platform.claude.com/docs/en/build-with-claude/str

OFFICIAL Anthropic Computer Use Tool — platform.claude.com/docs/en/agents-and-tools/tool-

OFFICIAL Win32 SetLayeredWindowAttributes — learn.microsoft.com/en-us/windows/win32/api/w

OFFICIAL DWM Performance Best Practices — learn.microsoft.com/en-us/windows/win32/dwm/bes

OFFICIAL Python ctypes documentation — docs.python.org/3/library/ctypes.html

OFFICIAL pyttsx3 Official Documentation — pyttsx3.readthedocs.io/en/latest/

Tier 2: Industry Practitioners

INDUSTRY Simon Willison — Claude Computer Use explorations (simonwillison.net)

INDUSTRY Claude Vision Object Detection — github.com/Doriandarko/Claude-Vision-Object-Det

INDUSTRY Anthropic Claude Cookbook: Extracting Structured JSON — github.com/anthropics/an

INDUSTRY pythonguis.com — PyQt6 QPropertyAnimation Tutorial

INDUSTRY Instructor: Structured Outputs with Anthropic — python.useinstructor.com

INDUSTRY Creating Spotlight Tutorials in Flutter — dev.to (DEV Community)

Tier 4: Source Code / Open Source

SOURCE CODE pyqt-transparent-window — github.com/yjg30737/pyqt-transparent-window

SOURCE CODE Python-ScreenOverlay — github.com/codesidian/Python-ScreenOverlay

SOURCE CODE ctypes SetWindowLongPtr sample — gist.github.com/ousttrue/1524707

SOURCE CODE PyQt widget overlay — gist.github.com/zhanglongqi

Tier 5: Community Resources

COMMUNITY Qt Forum: Click-through windows — forum.qt.io/topic/83161

COMMUNITY Qt Forum: QPainter glow effect — forum.qt.io/topic/129515

COMMUNITY Qt Forum: Pulsing image animation — forum.qt.io/topic/122352

COMMUNITY Qt Forum: Click-through blink fix — forum.qt.io/topic/156799

COMMUNITY Glowcast App (macOS annotation tool) — glowcastapp.com

— End of Document —

Sworder:721 | Main Objective: AAA Studio Quality | 2026-03-29